SmartLogistics — Execution Guidelines

1. Objective of This Assignment

This assignment is designed as a structured learning journey for engineers.

The intention is not only to build features, but to:

- Understand distributed logistics systems
- Learn event-driven architecture patterns
- Design for scalability and reliability
- Gain workflow orchestration experience
- Apply GenAI in operational systems
- Improve engineering system thinking

2. Execution Approach

Module-Based Progression

The assignment is divided into weekly learning modules.

To maximize learning:

- Complete each module in sequence
- Share progress with mentor regularly
- Incorporate feedback before next stage
- Build strong foundations before advanced modules

Weekly Milestone Flow

Each week should ideally follow this cycle:

- Work on assigned scope
- Submit implementation for review
- Discuss technical decisions
- Improve based on feedback
- Continue to next milestone

Mentor Collaboration

During reviews:

- Explain architecture choices

- Discuss tradeoffs
- Highlight blockers
- Ask questions openly

Reviews should focus on:

- System thinking
- Design clarity
- Tool understanding
- Implementation quality

3. Submission Guidelines

GitHub Repository

Maintain:

- Clean project structure
- Logical module separation
- Meaningful commits
- Good code organization

README

Include:

- Project overview
- Architecture diagram
- Setup instructions
- API overview
- Technology stack used

Technical Documentation

Include:

- Service/module breakdown
- Data flows
- Event flows
- Key design decisions
- Assumptions and tradeoffs

4. Learning Expectations

Part A (Core Platform)

Focus on:

- Event-driven architecture
- Idempotency
- Inventory consistency
- Workflow orchestration
- Async systems
- Observability basics

Part B (GenAI Layer)

Focus on:

- Embeddings and vector search
- Retrieval-Augmented systems
- Prompt engineering
- Response quality
- Streaming and latency handling

5. Part A — Execution Plan (3 Weeks)

Focus: Logistics Core + Distributed Systems

🟢 Week 1 — Foundation + Core Services

Scope

- Initial architecture and project setup
- Shipment management APIs
- User roles and auth basics
- Warehouse schema design
- Local environment setup

Deliverables

- Working APIs
- Database schema ready
- Docker setup operational
- Clean service structure established

🟡 Week 2 — Dispatch + Delivery Workflow

Scope

- Dispatch workflow using Temporal
- Courier assignment flow
- Tracking lifecycle handling
- Duplicate prevention
- State transitions and validations

Deliverables

- Stable dispatch workflow
- Courier orchestration ready
- Reliable tracking state machine

🔴 Week 3 — Event-Driven System + Observability

Scope

- Kafka integration
- Celery background workers
- Notification flows
- Basic analytics metrics
- Logs, tracing, dashboards

Deliverables

- Event-driven workflows operational
- Monitoring dashboards ready
- Distributed tracing enabled

Part A — Weekly Tracking Table

Week	Focus Area	What to Aim For	Done
Week 1	Foundation & Core Services	Working APIs, schema, setup	☐
Week 2	Dispatch & Workflows	Stable dispatch + tracking workflows	☐
Week 3	Events & Observability	Async processing + operational visibility	☐

6. Part B — Execution Plan (2 Weeks)

Focus: GenAI + Intelligent Logistics Layer

🟣 Week 4 — Data Preparation + Retrieval

Scope

- Shipment and ops data chunking
- Embedding generation
- Vector database integration
- Semantic retrieval system

Deliverables

- Searchable logistics embeddings
- Relevant retrieval pipeline working

🔵 Week 5 — AI Assistant + Streaming

Scope

- Operations AI assistant
- Delay insight engine
- Courier recommendation engine
- Streaming responses
- AI monitoring metrics

Deliverables

- Functional AI assistant
- Context-aware responses
- Smooth operational experience

Part B — Weekly Tracking Table

Week	Focus Area	What to Aim For	Done
Week 4	Retrieval Layer	Indexed operational data + semantic retrieval	☐
Week 5	AI Logistics Layer	Assistant + optimization tools	☐

7. Final Deliverables Checklist

Before completion ensure:

- ☐ All milestones completed and reviewed
- ☐ Codebase is clean and maintainable
- ☐ README and documentation completed
- ☐ Key workflows tested end-to-end
- ☐ Architecture decisions clearly explainable

8. Evaluation Approach

Evaluation will be based on:

- Architecture clarity
- Code quality and maintainability
- Correct use of defined tech stack
- Reliability and failure handling
- Observability maturity
- Depth of engineering understanding